Rozwiązywanie problemów — TestGEN

Spis treści

- Rozwiązywanie problemów — indeks
  - Dokumenty
  - Powiązane
- Przewodnik rozwiązywania problemów TestGEN
  - Spis treści
  - 1. Diagnostyka
  - 2. Analiza zawiesza się / timeout
  - 3. Upload pliku nie działa
  - 4. Błędy autoryzacji
  - 5. Wolna wydajność
  - 6. Błędy integracji
  - 7. Błędy płatności
  - 8. Problemy z RLS / izolacja danych
  - 9. FAQ techniczne
  - 10. Przewodnik eskalacji
  - Powiązane dokumenty
- Runbook: weryfikacja integracji tpay na środowisku testowym (sandbox)
  - 0. Warunki wstępne (jednorazowo)
  - 1. Zasada sterowania scenariuszami: KWOTA = przełącznik sandboxa
  - 2. Dane płatnicze sandbox (Panel Transakcyjny tpay)
  - 3. Scenariusze
  - 4. Weryfikacje SQL / przyspieszanie czasu
  - 5. Czyszczenie stanu między scenariuszami
  - 6. Kryteria zaliczenia (całość)

Rozwiązywanie problemów — indeks

Dokumenty

Dokument	Zakres
troubleshooting.md	**Przewodnik rozwiązywania problemów:** diagnostyka (Loki/Grafana), typowe awarie (analiza, upload, auth, wydajność, integracje, płatności, RLS), 33 FAQ, eskalacja L1→L2→L3
tpay-sandbox-runbook.md	Runbook środowiska testowego tpay: 14 scenariuszy GUI, kwoty sterujące, weryfikacja webhooków i faktur

Powiązane

- Webhooki tpay — monitoring Prometheus, health, replay
- Zadania okresowe — odnowienia, progi limitów
- Środowisko — konfiguracja zmiennych i serwisów

Przewodnik rozwiązywania problemów TestGEN

> **Wersja:** 1.0 · **Ostatnia aktualizacja:** 2026-09 · **Grupa docelowa:** Administrator systemu (L1 support)

Spis treści

1. Diagnostyka
2. Analiza zawiesza się / timeout
3. Upload pliku nie działa
4. Błędy autoryzacji
5. Wolna wydajność
6. Błędy integracji
7. Błędy płatności
8. Problemy z RLS / izolacja danych
9. FAQ techniczne
10. Przewodnik eskalacji

---

1. Diagnostyka

1.1. Logi — Loki / Grafana Explore

Dostęp: Grafana → Explore → Data source: Loki

Logi TestGEN mają format JSON ze standardowymi polami:

- `level` — error, warning, info, debug
- `request_id` — unikalny identyfikator żądania (UUID)
- `tenant_id` — firma, której dotyczy operacja
- `user_id` — użytkownik wykonujący operację
- `message` — treść komunikatu

Podstawowe zapytania LogQL:

```
# Wszystkie błędy z ostatniej godziny
{app="testgen-backend"} | json | level="error"

# Logi konkretnego żądania
{app="testgen-backend"} | json | request_id="<uuid>"

# Logi konkretnej firmy
{app="testgen-backend"} | json | tenant_id="<uuid>"

# Logi konkretnego użytkownika
{app="testgen-backend"} | json | user_id="<uuid>"

# Częstotliwość błędów (okno 5 min)
sum(rate({app="testgen-backend"} | json | level="error" [5m]))

# Logi Celery workera
{app="testgen-celery-worker"} | json | level=~"error|warning"

# Błędy płatności
{app="testgen-backend"} | json |
message=~".*payment.*|.*webhook.*|.*billing.*"
```

1.2. Metryki — Grafana Dashboard

Dashboard „TestGEN — przegląd produkcyjny" (`testgen-overview`):

Panel	Co sprawdzić	Próg ostrzegawczy
Wykorzystanie CPU	Czy któryś komponent > 70%?	> 80%
Wykorzystanie pamięci	Czy któryś komponent > 75%?	> 85%
Wykorzystanie dysku	Czy wolne miejsce < 20%?	< 15%
Żądania HTTP/s	Czy są anomalie w ruchu?	Nagły wzrost/spadek
Czas odpowiedzi P95	Czy > 2 s?	> 5 s
Współczynnik błędów	Czy > 1%?	> 5%
Kolejki RabbitMQ	Czy głębokość > 500?	> 1000
Celery zadania	Czy są zaległe zadania?	> 100
PostgreSQL połączenia	Czy > 150?	> 190 (z 200)
Redis pamięć	Czy zbliża się do maxmemory?	> 80%

1.3. Status komponentów

Docker Compose:

```
# Status kontenerów
docker compose ps

# Logi komponentu (ostatnie 100 linii)
docker compose logs --tail=100 backend
docker compose logs --tail=100 celery-worker

# Health check
docker compose exec backend curl -s localhost:8000/api/v1/health

# Restart komponentu
docker compose restart celery-worker
```

Kubernetes:

```
# Status podów
kubectl get pods -n testgen
kubectl get pods -n testgen-data

# Szczegóły poda z błędami
kubectl describe pod <pod-name> -n testgen

# Logi
kubectl logs <pod-name> -n testgen --tail=100
kubectl logs <pod-name> -n testgen --previous  # poprzednia instancja
         (crash)

# Health
kubectl exec -n testgen <backend-pod> -- curl -s
         localhost:8000/api/v1/health
```

PostgreSQL:

```
# Sprawdzenie dostępności
pg_isready -h localhost -p 5432

# Aktywne połączenia
psql -c "SELECT count(*) FROM pg_stat_activity WHERE datname='testgen';"

# Długo trwające zapytania (> 30 s)
psql -c "SELECT pid, now()-query_start AS duration, query
         FROM pg_stat_activity
         WHERE state='active' AND now()-query_start > interval '30
         seconds';"
```

Redis:

```
redis-cli ping          # Oczekiwane: PONG
redis-cli INFO memory   # Zużycie pamięci
redis-cli INFO clients  # Liczba połączeń
redis-cli DBSIZE        # Liczba kluczy
```

RabbitMQ:

```
rabbitmqctl status
rabbitmqctl list_queues name messages consumers  # Kolejki + głębokość
# Lub Management UI: http://localhost:15672 (guest/guest)
```

---

2. Analiza zawiesza się / timeout

Symptomy

- Analiza w stanie „W toku" powyżej 30 minut
- Brak wyników w projekcie
- Pasek postępu się nie rusza
- Odpowiedź 504 Gateway Timeout

Przyczyny

1. **Martwy Celery worker** — proces się zawiesił lub został zabity (OOM kill)
2. **Przepełniona kolejka** — zbyt wiele zadań w kolejce `analysis`
3. **Timeout API LLM** — dostawca LLM nie odpowiada (tryb `llm` / `hybrid`)
4. **Brak pamięci** — worker zabity przez kernel OOM killer
5. **Blokada bazy danych** — długo trwająca transakcja blokuje zapis

Rozwiązanie krok po kroku

1. **Sprawdź status Celery workerów:**

   ```
   # Docker Compose
   docker compose exec celery-worker celery -A app.workers.celery_app
            inspect active

   # Kubernetes
   kubectl exec -n testgen <worker-pod> -- celery -A
            app.workers.celery_app inspect active
   ```

   Jeśli brak odpowiedzi → worker martwy.

2. **Sprawdź głębokość kolejek:**

   ```
   rabbitmqctl list_queues name messages consumers
   ```

   Jeśli `analysis` > 1000 i consumers = 0 → workerzy nie konsumują.

3. **Sprawdź logi workera:**

   ```
   docker compose logs --tail=200 celery-worker | grep -i
            "error\|killed\|oom"
   ```

4. **Sprawdź metryki CPU/RAM:** Grafana → Dashboard → panel „Wykorzystanie pamięci". Jeśli worker > 90% RAM → OOM.

5. **Restart workera:**

```
docker compose restart celery-worker
# lub K8s:
kubectl rollout restart deployment celery-worker -n testgen
```

6. **Jeśli problem z LLM API:**

```
{app="testgen-celery-worker"} | json |
message=~".*timeout.*|.*LLM.*|.*anthropic.*|.*openai.*"
```

   Sprawdź status dostawcy LLM. Rozważ przełączenie na tryb `native`.

Zapobieganie

- Monitoring głębokości kolejek (alert > 1000)
- Auto-scaling workerów w K8s (HPA: queue depth > 100)
- Timeout LLM: domyślnie 60 s (`LLM_TIMEOUT`)
- Limit pamięci per worker: `--max-memory-per-child=500000` (Celery)

---

3. Upload pliku nie działa

Symptomy

- Błąd przy przesyłaniu pliku (komunikat w UI)
- Plik nie pojawia się w projekcie
- HTTP 413 Request Entity Too Large
- HTTP 507 Insufficient Storage

Przyczyny

1. **MinIO niedostępne** — kontener nie działa lub dysk pełny
2. **Przekroczony limit rozmiaru** — nginx/ingress ogranicza rozmiar żądania
3. **Nieobsługiwany format** — plik w formacie spoza listy dozwolonych
4. **Brak miejsca na dysku** — wolumen MinIO pełny
5. **Przekroczony limit per plan** — za dużo plików na analizę

Rozwiązanie krok po kroku

1. **Sprawdź status MinIO:**

```
docker compose exec minio mc admin info local
# lub
curl -s http://localhost:9000/minio/health/live
```

2. **Sprawdź miejsce na dysku:**

```
docker compose exec minio df -h /data
```

3. **Sprawdź limity nginx:**

```
# W nginx.conf — sekcja server
client_max_body_size 100m;  # Powinno być >= 100 MB
```

Dla K8s Ingress:

```
nginx.ingress.kubernetes.io/proxy-body-size: "100m"
```

4. **Sprawdź format pliku:** Obsługiwane formaty kodu: `.py`, `.java`, `.js`, `.ts`, `.cs`, `.c`, `.cpp`, `.php`, `.go`, `.rb` Obsługiwane formaty dokumentów: `.pdf`, `.docx`, `.html`, `.md`, `.txt` Obsługiwane formaty UML: `.xmi`, `.puml`, `.plantuml`, `.svg` Obsługiwane formaty GUI: `.png`, `.jpg`, `.jpeg`

5. **Sprawdź limity per plan:**
   - Freemium: 5 plików na analizę, 0,5 GB storage
   - Basic: 25 plików, 5 GB
   - Premium: 50 plików, 10 GB

Zapobieganie

- Monitoring dysku MinIO (alert < 15% wolnego)
- Polityka lifecycle: automatyczne czyszczenie plików tymczasowych (> 30 dni)
- Jasna komunikacja limitów w UI

---

4. Błędy autoryzacji

Symptomy

- HTTP 401 Unauthorized — „Token expired" lub „Invalid token"
- HTTP 403 Forbidden — „Insufficient permissions"
- Nagłe wylogowanie użytkownika
- Pusty dashboard po logowaniu

Przyczyny

1. **JWT wygasł** — domyślny czas życia: 15 min (access), 7 dni (refresh)
2. **Clock skew** — różnica czasu między serwerami > 60 s
3. **Token unieważniony** — konto zablokowane lub zmieniona rola
4. **Błędna polityka RLS** — brak kontekstu `app.current_tenant`
5. **Brak uprawnień roli** — użytkownik z rolą `user` próbuje operacji `owner`

Rozwiązanie krok po kroku

1. **Sprawdź czas serwera:**

   ```
   date -u                    # Czas serwera
   timedatectl status         # Synchronizacja NTP
   ```

   Jeśli NTP nie zsynchronizowany:

   ```
   sudo timedatectl set-ntp true
   ```

2. **Dekoduj JWT (bez weryfikacji podpisu):**

   ```
   # Wyciągnij payload z tokena
   echo "<token>" | cut -d. -f2 | base64 -d 2>/dev/null | python3 -m
            json.tool
   ```

   Sprawdź: `exp` (wygaśnięcie), `iat` (wystawienie), `tenant_id`, `role`, `is_admin`.

3. **Sprawdź stan konta użytkownika:**

   ```
   SELECT id, email, is_active, is_blocked, deleted_at
   FROM users WHERE email = '<email>';
   ```

4. **Sprawdź polityki RLS:**

   ```
   SELECT tablename, policyname, cmd, qual
   FROM pg_policies WHERE schemaname = 'public';
   ```

5. **Sprawdź rolę w firmie:**

   ```
   SELECT ut.role, t.name AS tenant_name
   FROM user_tenants ut
   JOIN tenants t ON t.id = ut.tenant_id
   WHERE ut.user_id = '<user_uuid>';
   ```

Zapobieganie

- Synchronizacja NTP na wszystkich serwerach
- Automatyczne odświeżanie tokenów (refresh token flow)
- Monitoring nieudanych logowań (alert > 10/min z jednego IP)
- Testy automatyczne RLS przy każdym deploy'u

---

5. Wolna wydajność

Symptomy

- Czas odpowiedzi API > 5 s
- Wolne ładowanie dashboardu
- Timeout przy generowaniu raportów
- UI „zamraża się" na kilka sekund

Przyczyny

1. **CPU/RAM wyczerpane** — zbyt dużo równoległych analiz
2. **Connection pool wyczerpany** — PgBouncer / asyncpg brak wolnych połączeń
3. **Wolne zapytania SQL** — brak indeksów, full table scan
4. **Redis pełny** — maxmemory osiągnięty, eviction policy
5. **Sieć** — problemy z łącznością między kontenerami

Rozwiązanie krok po kroku

1. **Sprawdź metryki CPU/RAM:** Grafana → panel „Wykorzystanie CPU" i „Wykorzystanie pamięci". Jeśli backend > 80% CPU → rozważ dodanie replik.

2. **Sprawdź połączenia PostgreSQL:**

```sql
-- Aktywne połączenia per stan
SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'testgen'
GROUP BY state;

-- Długo trwające zapytania
SELECT pid, now()-query_start AS duration, left(query, 100)
FROM pg_stat_activity
WHERE state = 'active'
  AND now()-query_start > interval '10 seconds'
ORDER BY duration DESC;
```

3. **Sprawdź PgBouncer (jeśli używany):**

```
psql -p 6432 pgbouncer -c "SHOW POOLS;"
psql -p 6432 pgbouncer -c "SHOW STATS;"
```

4. **Zidentyfikuj wolne zapytania:**

```sql
-- Top 10 najwolniejszych zapytań (pg_stat_statements)
SELECT left(query, 80), calls, mean_exec_time::int AS avg_ms,
       total_exec_time::int AS total_ms
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

5. **Sprawdź brakujące indeksy:**

```sql
SELECT relname, seq_scan, idx_scan
FROM pg_stat_user_tables
WHERE seq_scan > 1000 AND idx_scan < 100
ORDER BY seq_scan DESC;
```

6. **Sprawdź Redis:**

```
redis-cli INFO memory   # used_memory vs maxmemory
redis-cli INFO stats    # evicted_keys (powinno być 0)
```

Zapobieganie

- Connection pooling (PgBouncer, `pool_mode=transaction`)
- Regularne `VACUUM ANALYZE` (autovacuum domyślnie włączony)
- Monitorowanie slow queries (> 1 s)
- Auto-scaling w K8s (HPA: CPU > 70%)
- Redis: adekwatne `maxmemory` + polityka `allkeys-lru`

---

6. Błędy integracji

Symptomy

- Synchronizacja z GitHub/GitLab nie działa
- Eksport do TestRail zwraca błąd
- Tworzenie zadań w Jira nie powodzi się
- Komunikat „Integration connection failed"

Przyczyny

1. **Token wygasł** — token dostępu do systemu zewnętrznego utracił ważność
2. **Rate limit** — przekroczony limit żądań do API zewnętrznego
3. **Zmienione API** — system zewnętrzny zmienił wersję API
4. **Problemy z siecią** — firewall, DNS, proxy blokują połączenie
5. **Błędna konfiguracja** — niepoprawny URL, project key, workspace

Rozwiązanie krok po kroku

1. **Sprawdź konfigurację integracji:** W UI: Projekt → Ustawienia → Integracje → sprawdź status połączenia.

2. **Sprawdź token dostępu:**

```
# Test połączenia z GitHub
curl -H "Authorization: token <ghp_...>" https://api.github.com/user

# Test połączenia z Jira
curl -u email:api_token
         https://<instance>.atlassian.net/rest/api/3/myself
```

3. **Sprawdź rate limits:**

```
# GitHub — nagłówki odpowiedzi
curl -I -H "Authorization: token <ghp_...>"
         https://api.github.com/rate_limit
# X-RateLimit-Remaining powinno być > 0
```

4. **Sprawdź logi integracji:**

```
{app="testgen-backend"} | json |
message=~".*integration.*|.*jira.*|.*github.*"
```

5. **Sprawdź połączenie sieciowe:**

```
docker compose exec backend curl -s -o /dev/null -w "%{http_code}"
         https://api.github.com
# Oczekiwane: 200 lub 403 (brak tokena)
```

Zapobieganie

- Rotacja tokenów co 90 dni
- Monitoring statusu integracji (health check co 5 min)
- Obsługa rate limitów: retry z backoff
- Dokumentacja wymaganych uprawnień per integracja

---

7. Błędy płatności

Symptomy

- Płatność nie przechodzi — komunikat błędu na stronie tpay
- Plan nie aktualizuje się po płatności
- Webhook tpay nie dochodzi
- Brak faktury po udanej płatności
- Firma w stanie „grace" mimo zapłaty

Przyczyny

1. **tpay niedostępne** — przerwa w działaniu bramki płatności
2. **Webhook nie dostarczony** — problem z siecią lub niepoprawny URL zwrotny
3. **Błędny podpis JWS** — webhook odrzucony przez weryfikację podpisu
4. **Karta odrzucona** — niewystarczające środki, karta wygasła
5. **Duplikat transakcji** — idempotency conflict (HTTP 409)

Rozwiązanie krok po kroku

1. **Sprawdź status tpay:** Odwiedź stronę statusu tpay lub sprawdź panel merchantów.

2. **Sprawdź logi webhooków:**

```
{app="testgen-backend"} | json |
message=~".*webhook.*|.*tpay.*|.*payment.*"
```

3. **Sprawdź zapisane zdarzenia webhooków:**

```
SELECT id, event_type, status, created_at
FROM webhook_events
WHERE provider = 'tpay'
ORDER BY created_at DESC
LIMIT 20;
```

4. **Sprawdź status płatności:**

```
SELECT id, status, amount, created_at
FROM payments
WHERE tenant_id = '<tenant_uuid>'
ORDER BY created_at DESC
LIMIT 10;
```

5. **Replay webhooka** (jeśli zdarzenie zapisane, ale nie przetworzone):

```
curl -X POST
         http://localhost:8000/api/v1/admin/billing/webhooks/<event_u
         id>/replay \
  -H "Cookie: <admin_jwt>"
```

6. **Ręczna aktywacja planu** (ostateczność):

```
UPDATE subscriptions
SET plan = 'basic', status = 'active',
    current_period_end = now() + interval '30 days'
WHERE tenant_id = '<tenant_uuid>';
```

> **Ostrzeżenie:** Ręczna zmiana planu omija system bilingowy. Użyj tylko w sytuacjach awaryjnych i udokumentuj w audycie.

Zapobieganie

- Monitoring dostarczalności webhooków (Prometheus metryka)
- Automatyczne retry webhooków (tpay ponawia 3× w ciągu 48 h)
- Alert na firmy w stanie `grace` dłużej niż 3 dni
- Regularna weryfikacja klucza publicznego tpay (do weryfikacji JWS)

Szczegóły: Runbook tpay sandbox, Architektura webhooków

---

8. Problemy z RLS / izolacja danych

Symptomy

- Użytkownik widzi dane innej firmy (krytyczne!)
- Puste wyniki mimo istniejących danych
- Błąd RLS policy violation
- Zapytanie zwraca 0 rekordów choć powinno zwrócić więcej

Przyczyny

1. **Brak kontekstu sesji** — `app.current_tenant` nie ustawiony
2. **Błędny tenant_id w JWT** — token zawiera niepoprawny identyfikator firmy
3. **Uszkodzona polityka RLS** — brakująca lub niepoprawna polityka na tabeli
4. **RLS wyłączony na tabeli** — ktoś wyłączył RLS ręcznie
5. **Superuser bypass** — połączenie jako superuser pomija RLS

Rozwiązanie krok po kroku

1. **Sprawdź kontekst sesji:**

```
-- Na sesji, która ma problem
SELECT current_setting('app.current_tenant', true);
-- Powinno zwrócić UUID firmy; NULL = brak kontekstu
```

2. **Sprawdź, czy RLS aktywny na tabelach:**

```
SELECT tablename, rowsecurity
FROM pg_tables
WHERE schemaname = 'public' AND rowsecurity = true
ORDER BY tablename;
```

Kluczowe tabele z RLS: `projects`, `source_files`, `analysis_results`, `test_cases`, `test_code`, `security_findings`, `gap_findings`, `invoices`, `subscriptions`.

3. **Sprawdź polityki RLS:**

```
SELECT tablename, policyname, cmd, qual
FROM pg_policies
WHERE schemaname = 'public'
ORDER BY tablename;
```

4. **Test izolacji:**

```
-- Ustaw kontekst na firmę A
SET LOCAL app.current_tenant = '<tenant_a_uuid>';
SELECT count(*) FROM projects;  -- Powinno zwrócić tylko projekty
         firmy A

-- Zmień na firmę B
SET LOCAL app.current_tenant = '<tenant_b_uuid>';
SELECT count(*) FROM projects;  -- Tylko projekty firmy B
```

5. **Jeśli użytkownik widzi cudze dane — natychmiastowa eskalacja do L3!**

Zapobieganie

- Automatyczne testy RLS przy każdym deploy'u (test per tabela)
- Nigdy nie łącz się jako superuser do produkcji (użyj roli `testgen_app`)
- Nigdy nie wyłączaj RLS na tabelach produkcyjnych
- Monitoring: alert na zapytania bez kontekstu `app.current_tenant`

> **Ostrzeżenie:** Wyciek danych między firmami to incydent bezpieczeństwa. Natychmiastowa eskalacja do L3 + powiadomienie zespołu bezpieczeństwa.

---

9. FAQ techniczne

Instalacja i konfiguracja

P1: Jakie są minimalne wymagania sprzętowe? Docker Compose (single-node): 4 vCPU, 16 GB RAM, 100 GB SSD. Kubernetes: min. 3 node'y, łącznie 12 vCPU, 48 GB RAM.

P2: Jak zmienić port aplikacji? Edytuj `docker-compose.yml` → sekcja `frontend` → `ports: "8080:80"` (zamiast `4200:80`). Zaktualizuj również `nginx.conf`.

P3: Jak dodać certyfikat SSL? Umieść pliki `cert.pem` i `key.pem` w katalogu certyfikatów i wskaż je w konfiguracji nginx. Szczegóły: Dokumentacja wdrożeniowa — SSL/TLS.

P4: Jak zmienić hasło administratora? Użyj endpointu API: `POST /api/v1/auth/reset-password` z tokenem admina. Lub ręcznie: zhaszuj nowe hasło (Argon2id) i zaktualizuj `users.password_hash` w bazie.

P5: Jak sprawdzić wersję systemu? `GET /api/v1/health` zwraca `version` w odpowiedzi. Lub: `docker compose exec backend python -c "from app.core.config import settings; print(settings.app_version)"`.

Baza danych

P6: Jak uruchomić migrację bazy danych?

```
docker compose exec backend alembic upgrade head
```

P7: Jak cofnąć ostatnią migrację?

```
docker compose exec backend alembic downgrade -1
```

P8: Jak sprawdzić stan migracji?

```
docker compose exec backend alembic current
docker compose exec backend alembic history --verbose
```

P9: Jak zrobić backup bazy danych?

```
docker compose exec postgres pg_dump -U testgen testgen > backup.sql
```

Lub użyj skryptu `docs/final/deployment-guide/examples/backup-postgres.sh`.

P10: Dlaczego zapytania są wolne po imporcie danych? Uruchom `VACUUM ANALYZE;` na bazie — odświeży statystyki planera zapytań i odzyska miejsce.

Moduły AI

P11: Jak zmienić domyślny tryb silnika (engine_mode)? Panel admina → Konfiguracja → sekcja LLM. Lub zmienna `ENGINE_MODE=native|llm|hybrid`.

P12: Co zrobić gdy API LLM nie odpowiada? Sprawdź status dostawcy (status.anthropic.com lub status.openai.com). System automatycznie ponawia próbę 3× z backoff. Rozważ przełączenie na `native`.

P13: Jak zmienić dostawcę LLM? Zmienna `LLM_PROVIDER=anthropic|openai`. Ustaw odpowiedni klucz API: `ANTHROPIC_API_KEY` lub `OPENAI_API_KEY`.

P14: Ile kosztuje analiza w trybie LLM? Estymacja: ~0,05–0,50 PLN per analiza (zależy od rozmiaru kodu i modelu). Monitoruj w tabeli `llm_usage`.

P15: Jak sprawdzić zużycie limitu kosztów LLM?

```
SELECT tenant_id, sum(cost_usd) AS total_cost
FROM llm_usage
WHERE created_at >= date_trunc('month', now())
GROUP BY tenant_id;
```

Integracje

P16: Jak skonfigurować integrację z Jira? Projekt → Ustawienia → Integracje → Jira. Wymagane: URL instancji, e-mail, API token, klucz projektu.

P17: Jak skonfigurować integrację z Azure DevOps? Projekt → Ustawienia → Integracje → Azure DevOps. Wymagane: URL organizacji, PAT (Personal Access Token), nazwa projektu.

P18: Dlaczego GitHub webhook nie działa? Sprawdź: 1) URL webhooka (musi być dostępny z internetu), 2) secret (musi zgadzać się z konfiguracją), 3) zdarzenia (push, pull_request).

P19: Jak wyeksportować testy do TestRail? Projekt → Testy → Eksport → TestRail CSV. Plik CSV zgodny z formatem importu TestRail.

P20: Jakie uprawnienia potrzebuje token GitHub? Minimalnie: `repo` (private repos) lub `public_repo` (public). Dla webhooków: `admin:repo_hook`.

Wydajność

P21: Jak dodać więcej workerów Celery? Docker Compose: `docker compose up -d --scale celery-worker=4`. K8s: `kubectl scale deployment celery-worker -n testgen --replicas=4`.

P22: Jak zwiększyć connection pool PostgreSQL? PgBouncer: edytuj `pgbouncer.ini` → `max_client_conn=400`, `default_pool_size=50`. Bezpośrednio: `postgresql.conf` → `max_connections=300`.

P23: Jak wyczyścić cache Redis?

```
redis-cli FLUSHDB          # Tylko bieżąca baza
redis-cli FLUSHALL         # Wszystkie bazy (uwaga!)
```

P24: Dlaczego generowanie raportu PDF jest wolne? WeasyPrint (generowanie PDF) jest CPU-intensive. Dla dużych raportów (> 100 stron) może trwać 30–60 s. Zadanie wykonywane w Celery (kolejka `export`).

P25: Jak monitorować wydajność bazy? Zainstaluj `pg_stat_statements`: `CREATE EXTENSION pg_stat_statements;`. Dashboard Grafana: PostgreSQL metrics.

Bezpieczeństwo

P26: Jak wymusić MFA dla wszystkich użytkowników firmy? Ustawienia firmy → Bezpieczeństwo → Wymuś MFA. Użytkownicy bez TOTP zostaną poproszeni o konfigurację przy następnym logowaniu.

P27: Jak sprawdzić logi audytu? Panel admina → Dziennik audytu (z filtrami). Lub SQL:

```
SELECT * FROM audit_logs
WHERE tenant_id = '<uuid>' ORDER BY created_at DESC LIMIT 50;
```

P28: Jak zrotować klucz JWT? Wygeneruj nową parę kluczy RSA, zaktualizuj zmienne `JWT_PRIVATE_KEY` i `JWT_PUBLIC_KEY`, zrestartuj backend. Istniejące tokeny wygasną naturalnie (max 7 dni).

P29: Jak zablokować adres IP? Na poziomie nginx: dodaj `deny <ip>;` w konfiguracji. Na poziomie firewall: `ufw deny from <ip>`.

P30: Czy dane są szyfrowane w spoczynku? PostgreSQL: szyfrowanie na poziomie dysku (LUKS/dm-crypt). MinIO: szyfrowanie SSE-S3. Redis: nie domyślnie — włącz `requirepass` i szyfrowanie TLS.

Dodatkowe

P31: Jak zmienić język interfejsu? TestGEN jest dostępny wyłącznie w języku polskim.

P32: Jak zresetować dane demonstracyjne?

```
docker compose exec backend python -m app.seed.run_seed
```

P33: Jak sprawdzić licencję? `GET /api/v1/health` zwraca informacje o licencji i planie.

---

10. Przewodnik eskalacji

10.1. Poziomy wsparcia

Poziom	Kto	Narzędzia	Zakres
L1 — Administrator	Admin platformy	Ten przewodnik, panel admina, Grafana	Diagnostyka, restart, typowe problemy
L2 — DevOps	Inżynier DevOps	Dokumentacja wdrożeniowa, kubectl, psql	Konfiguracja, skalowanie, backup/restore, upgrade
L3 — Zespół deweloperski	Deweloper	Kod źródłowy, debugger, logi aplikacji	Bugfix, RCA, hotfix

10.2. Kiedy eskalować

Kryterium	L1 → L2	L2 → L3
Czas trwania	Problem > 30 min po standardowej diagnostyce	Problem > 2 h po interwencji DevOps
Wpływ	> 10 użytkowników dotkniętych	Cała platforma niedostępna
Typ	Infrastruktura (dysk, sieć, certyfikaty)	Bug w kodzie, wyciek danych, RLS naruszenie
Dane	Podejrzenie utraty danych	Potwierdzona utrata lub wyciek danych
Płatności	Webhook nie działa > 1 h	Podwójne obciążenie użytkownika

10.3. Informacje do zebrania przed eskalacją

Przed eskalacją zbierz:

- ☐ **request_id** — identyfikator problematycznego żądania
- ☐ **timestamp** — dokładny czas wystąpienia (UTC)
- ☐ **tenant_id** — firma, której dotyczy problem
- ☐ **user_id** — użytkownik zgłaszający (jeśli dotyczy)
- ☐ **Logi** — fragment logów z Loki (filtr po request_id)
- ☐ **Metryki** — zrzut ekranu Grafana w momencie incydentu (CPU, RAM, error rate)
- ☐ **Kontekst** — co użytkownik robił, co próbowano, jakie kroki podjęto
- ☐ **Wpływ** — ilu użytkowników dotkniętych, które funkcje niedostępne

10.4. Szablon zgłoszenia incydentu

```
# Zgłoszenie incydentu

**Data/godzina:** YYYY-MM-DD HH:MM UTC
**Zgłaszający:** [imię, rola]
**Ważność:** Krytyczny / Wysoki / Średni / Niski

## Opis problemu
[Co się dzieje, jakie symptomy]

## Wpływ
- Dotknięte firmy: [lista tenant_id]
- Dotknięci użytkownicy: [liczba]
- Niedostępne funkcje: [lista]

## Podjęte kroki
1. [Co sprawdzono]
2. [Co zrobiono]
3. [Wynik]

## Dane diagnostyczne
- request_id: [uuid]
- Logi: [fragment lub link do Grafana]
- Metryki: [zrzut ekranu]

## Załączniki
[Logi, zrzuty ekranu, komendy i wyniki]
```

10.5. Post-incident (po rozwiązaniu)

1. **Komunikacja** — powiadom dotkniętych użytkowników o rozwiązaniu
2. **Root Cause Analysis (RCA)**:
   - Co się stało (timeline)
   - Dlaczego się stało (przyczyna główna)
   - Jak wykryto (monitoring, zgłoszenie użytkownika)
   - Jak rozwiązano (kroki)
   - Jak zapobiec powtórzeniu (action items)
3. **Action items** — wpisz zadania do backlogu (monitorowanie, alert, fix)
4. **Aktualizacja dokumentacji** — dodaj rozwiązanie do tego przewodnika jeśli problem był nowy

---

Powiązane dokumenty

- Dokumentacja wdrożeniowa — operacje — backup, skalowanie, monitoring, DR
- Podręcznik administratora — zarządzanie platformą
- Runbook tpay sandbox — scenariusze testowe płatności
- Architektura webhooków — monitoring i replay webhooków

Runbook: weryfikacja integracji tpay na środowisku testowym (sandbox)

> Story 7.12 — macierz scenariuszy płatności. Ten dokument to **instrukcja ręcznej weryfikacji na prawdziwym sandboxie tpay** (GUI + skrypty): pozytywne ścieżki ORAZ poprawna obsługa błędów. Odpowiednik automatyczny (mocki): `backend/tests/scenarios/` + `backend/tests/integration/test_tpay_sandbox.py` (marker `tpay_sandbox`). Dane sterujące: §2.1 planu integracji tpay (`_bmad-output/planning-artifacts/plan-integracji-tpay.md`).

0. Warunki wstępne (jednorazowo)

1. **Środowisko**: `https://testgen.nexio.pl` (serwer 10.0.3.22, Docker Compose), backend z env: `TPAY_ENV=sandbox` + `TPAY_CLIENT_ID` / `TPAY_CLIENT_SECRET` / `TPAY_SECURITY_CODE` / `TPAY_MERCHANT_ID`.
2. **Panel tpay (sandbox)** — Ustawienia → Powiadomienia:
   - URL notyfikacji: `https://testgen.nexio.pl/api/v1/billing/webhooks/tpay`
   - zaznaczone **„Zezwól na nadpisanie adresu URL powiadomienia"** (wymagane dla notyfikacji aliasów BLIK).
3. **Konto testowe**: tenant z rolą **Owner** (checkout/zmiana planu wymaga Ownera; nowy tenant startuje na Freemium — idealny punkt wyjścia).
4. **Nawigacja**: menu **Rozliczenia** w navbarze (Plany subskrypcji, Zużycie zasobów, Faktury; Owner dodatkowo: Dane fakturowe, Metody płatności).

1. Zasada sterowania scenariuszami: KWOTA = przełącznik sandboxa

Sandbox tpay reaguje na kwotę transakcji, a kwota checkoutu = cena planu z env (`BILLING_BASIC_MONTHLY_PRICE` / `BILLING_PREMIUM_MONTHLY_PRICE`). Skrypt `backend/scripts/set_sandbox_prices.py` podmienia cenę w `.env`; DB synchronizuje się seedem przy restarcie.

```
# na serwerze testowym (w katalogu z docker-compose; dostosuj ścieżkę .env)
docker compose exec backend python scripts/set_sandbox_prices.py --list   # dostępne scenariusze
docker compose exec backend python scripts/set_sandbox_prices.py --scenario frictionless
docker compose up -d backend            # restart = seed synchronizuje DB
# weryfikacja nowej ceny:
curl -s https://testgen.nexio.pl/api/v1/billing/plans -H "Authorization: Bearer <JWT>" | jq '.plans[].price_monthly'
```

Po każdej zmianie scenariusza: restart backendu + odśwież stronę planów (cena na karcie planu musi pokazywać kwotę sterującą). Po testach: `--reset` (Basic 30.00 / Premium 50.00).

> Kwoty sterujące NIE są cenami produktu (§3.2a) — to przełączniki testowe.

2. Dane płatnicze sandbox (Panel Transakcyjny tpay)

Co	Wartość	Uwagi
Kod BLIK	`777123` (dowolne `777xxx`)	poprawny kod
Karty	`2223 0002 8000 0016` (MC), `4056 2178 4359 7258` (Visa)	CVC: 3 dowolne cyfry; data: przyszła
Grosze 3DS	`.01` frictionless, `.03` sukces, `.05` błąd 3DS, `.06` decline, inne → challenge	kwota końcówka steruje
Błędy kart	`500.00` / `501.00` / `503.00` (+ `504.00` tylko token)	
Błędy BLIK	`144.00` płatnik, `168.00` wystawca, `120.00` brak środków, `312.00` timeout, `96.00` limit, `216.00` bezpieczeństwo	
Alias PAYID	grosze `.08` – `.15`	alias żyje ≥ tydzień; kody `999xxx` czyszczone co 15 min — nie polegać
Bank bez PP	`495.00`	refuseNoPayId → nasze 422

3. Scenariusze

Legenda weryfikacji: **GUI** (co widzi tester), **DB** (SQL — patrz §4), **API** (`curl`/strona), **EMAIL** — w sandboxie maile do płatników są zablokowane (§4.4 planu): potwierdzenia sprawdzamy w logach backendu (`docker compose logs backend | grep -i email` — żądania wysyłki) lub pomijamy.

Przed KAŻDYM scenariuszem zakupu tenant powinien być na **Freemium** (po porażce zostaje Freemium — można kontynuować; po sukcesie — patrz §5 reset).

S1. Zakup planu Basic kodem BLIK — pozytywna ścieżka (AC1)

Przygotowanie: `--scenario happy` (cena neutralna 30.00) + restart.

1. GUI: **Rozliczenia** → **Plany subskrypcji**; karta **Basic** → **Wybierz Basic**.
2. Strona „Przekierowanie do płatności" → automatyczny redirect na Panel Transakcyjny tpay (kwota 30.00, opis „TestGEN — subskrypcja Basic (miesięczna)").

3. Na panelu tpay: wybierz **BLIK**, wpisz `777123`, zatwierdź.
4. Powrót na `https://testgen.nexio.pl/billing/checkout/success?...` — strona potwierdzenia (polling statusu → „opłacona").

Oczekiwane (poprawność = FAIL, jeśli brak):

- GUI: plan aktualny = Basic (karta planów podświetla), liczniki w **Zużycie zasobów** liczą się od 0 (nowy okres).
- DB: `payment_transactions.status = 'paid'`, `test_mode = true`; `subscriptions.status = 'active'`, `next_charge_at = current_period_end`.
- Faktura: **Rozliczenia** → **Faktury** — pozycja z bieżącego dnia (PDF po chwile, generuje Celery; brak PDF = jeszcze nie wygenerowany, nie błąd).

S2. Zakup kartą z zapisem metody — pozytywna + token (AC2)

Przygotowanie: `--scenario happy` + restart; tenant Freemium.

1. **Rozliczenia** → **Plany subskrypcji** → **Wybierz Basic**.
2. Na panelu tpay: metoda **karta**, dane `2223 0002 8000 0016`, CVC `123`, data `12/30`; **zaznacz „zapamiętaj kartę"** (save=1).
3. (3DS challenge może się pojawić przy tej kwocie — potwierdź go.)
4. Powrót na stronę potwierdzenia.

Oczekiwane:

- **Rozliczenia** → **Metody płatności**: karta `•••• 0016 mastercard`, status aktywna, **domyślna** (odnowienia będą obciążać tę kartę).
- DB: `payment_tokens` (kind `card_token`, `tpay_status='active'`), `subscriptions.payment_token_id` ustawione.

S3. Zakup przelewem bankowym (PBL) — pozytywna (AC3)

Przygotowanie: `--scenario happy --plan premium` + restart.

1. Plany → **Wybierz Premium**; na panelu tpay: metoda **przelew**, wybierz dowolny bank z listy sandbox; opłać symulowanie.
2. Powrót na stronę potwierdzenia.

Oczekiwane: jak S1 (plan Premium, faktura). Uwaga: notyfikacja tpay nie niesie kanału płatności — „przelew" nie jest odrębnym polem w historii (ograniczenie u źródła, patrz README scenariuszy).

S4. 3DS sterowany groszami — 5 wariantów (AC4)

Dla każdego wariantu: ustaw scenariusz + restart + zakup Basic (S1 kroki), zapłać kartą `2223 0002 8000 0016`:

Wariant	Skrypt (`--scenario`)	Kwota	Na paneli tpay	Oczekiwany wynik w TestGEN
frictionless	`frictionless`	30.01	brak 3DS, od razu success	success, plan aktywny
sukces 3DS	`threeds_success`	30.03	challenge → potwierdź	success, plan aktywny
błąd 3DS	`threeds_error`	30.05	challenge → błąd	strona `/billing/checkout/error` z komunikatem + **przycisk ponowienia**; tenant nadal Freemium; nowy checkout możliwy
decline	`decline`	30.06	odmowa bez 3DS	jw.
challenge	`challenge`	30.02	challenge → potwierdź	success, plan aktywny

Obsługa błędów — asercje: strona błędu pokazuje status „nieopłacona", CTA ponowienia wraca na plany; DB: transakcja bez `paid`; subskrypcja niedotknięta (Freemium).

S5. Błędy kart — kwoty wymuszające (AC5)

Warianty: `card_error_500` / `card_error_501` / `card_error_503` (`card_error_504` dotyczy obciążenia tokenem — patrz S10). Krok jak S4 (karta, kwota sterująca): panel tpay zwraca błąd → strona błędu TestGEN + ponowienie. Tenant zostaje na planie wyjściowym.

S6. Błędy BLIK — 6 kwot (AC5)

Warianty: `blik_rejected_by_payer` (144), `blik_rejected_by_issuer` (168), `blik_no_funds` (120), `blik_timeout` (312), `blik_limit` (96), `blik_security` (216). Krok jak S1, ale BLIK `777123` na kwocie sterującej: odmowa → strona błędu + ponowienie; tenant Freemium; **powiadomienie o nieudanej płatności** widoczne w logach email.

S7. Bank bez Płatności Powtarzalnych — refuseNoPayId (AC10)

Przygotowanie: `--scenario refuse_no_payid` (495.00) + restart.

1. Plany → **Wybierz Basic**.

Oczekiwane: backend dostaje od tpay `refuseNoPayId` już przy tworzeniu transakcji → GUI pokazuje **422** z komunikatem PL: „Twój bank nie obsługuje płatności powtarzalnych BLIK… Zapłać kartą i zapisz ją jako metodę płatności…"

- powrót do planów. (Jeśli sandbox odmawia tylko przy opisie `mock`, potwierdź wariant API: `tests/integration/test_tpay_sandbox.py` → `test_bank_without_repeatable_payments_refuses_payid`.)

S8. Upgrade Basic → Premium z proration (AC6)

Przygotowanie: tenant na Basic (po S1), `--reset` + restart (ceny 30/50).

1. **Rozliczenia** → **Plany subskrypcji**; na karcie Premium akcja zmiany planu.
2. **Dialog potwierdzenia** zmiany planu pokazuje **kwotę prorated** (liczoną lokalnie: `((50-30)/dni okresu) × pozostałe dni`) — zweryfikuj z podglądem proration na tym samym dialogu (spójność preview↔charge).
3. Zapisana karta z S2 → obciążenie bez redirectu; inaczej checkout z kwotą prorated.
4. Po `paid`: plan Premium natychmiast.

Oczekiwane: GUI — aktualny plan Premium; **Zużycie zasobów** — limity Premium (analizy bez limitu), **liczniki zachowane** (nie zerują); DB: `plan_change_history` (upgrade, `completed`, `proration_details.amount`).

S9. Odnowienie aliasem BLIK PAYID — bez udziału użytkownika (AC10)

Przygotowanie: `--scenario payid_alias` (30.08) + restart; tenant Freemium.

1. Zakup Basic jak S1, ale na panelu tpay: BLIK `777123` **+ zaznacz zgodę na Płatności Powtarzalne (alias)**. Kwota 30.08 ∈ x.08–x.15 → alias zostaje zarejestrowany.
2. Weryfikacja: **Metody płatności** — pozycja „alias BLIK (PAYID)"; DB: `payment_tokens.kind = 'blik_alias_payid'`, status aktywny.
3. Wymuś wymagalność odnowienia (skróć okres — SQL §4.3), potem jednorazowy przebieg schedulera:

   ```
   docker compose exec backend python scripts/run_billing_scheduler.py
   ```

4. Na panelu tpay zatwierdź obciążenie merchant-initiated (sandbox może wymagać akceptacji płatnika) → webhook `paid`.

Oczekiwane: odnowienie bez logowania użytkownika do panelu TestGEN; DB: nowa transakcja `source=recurring`; `subscriptions` przedłużona o cykl (`current_period_start` = poprzedni `current_period_end`); faktura nowa.

Wariant błędu — wygaśnięcie aliasu: w Panelu tpay wyrejestruj alias (lub wyślij notyfikację testową `ALIAS_EXPIRED` narzędziem panelu): **Metody płatności** pokazują alias zawieszony; Owner dostaje powiadomienie email „dodaj kartę lub ponów BLIK" (logi); kolejny przebieg schedulera bez aktywnej metody → **dunning** (S11).

S10. Odnowienie zapisaną kartą (token) (AC9)

Przygotowanie: tenant z aktywną subskrypcją i kartą domyślną (po S2); `--reset` + restart.

1. Skróć `next_charge_at` (SQL §4.3) → `docker compose exec backend python scripts/run_billing_scheduler.py`.
2. Sandbox: obciążenie tokenem → `paid`.

Oczekiwane: jak S9 (przedłużenie, faktura, liczniki nowy okres). **Wariant błędu**: `--scenario card_error_504` + restart → przebieg schedulera → odmowa → **dunning** (próba 1; email „płatność nie powiodła się").

S11. Dunning: 3 próby → downgrade do Freemium + grace (AC12)

Przygotowanie: subskrypcja płatna z metodą; cena wymuszająca porażkę (np. `--scenario decline`); restart.

1. Uruchom `run_billing_scheduler.py` → porażka 1 (logi: „dunning (próba 1)", email do Ownera).
2. Skróć cooldown (SQL §4.3b) → przebieg → porażka 2; jw. → porażka 3.
3. Czwarty przebieg → **downgrade**: DB `status='past_due'`, `plan_id`=Freemium, `previous_plan_id`=poprzedni, `grace_until`≈+7 dni; email „subskrypcja obniżona".
4. **Grace**: limity planu dotychczasowego nadal obowiązują (Zużycie zasobów).
5. Po `grace_until` (SQL §4.3c przyspiesza) przebieg schedulera → egzekwja Freemium; **11. analiza** w miesiącu → **429 + banner „limit osiągnięty" z CTA upgrade** (GUI: uruchom analizę ponad limit).

S12. Downgrade / anulowanie / reaktywacja (AC7, AC8)

Przygotowanie: tenant na Premium; `--reset` + restart.

1. Plany → zmiana na Basic (downgrade): GUI potwierdza „na koniec okresu"; DB: `cancel_at_period_end=true`, `next_charge_at=NULL`; **asercja braku wywołań tpay** (logi backendu — żadnego żądania create/charge przy downgrade).
2. „Rezygnuj z zmiany" (reaktywacja): `cancel_at_period_end=false`, `next_charge_at` przywrócone; bez wywołań tpay.
3. Po `current_period_end` (SQL §4.3 przyspiesza) + przebieg schedulera: plan docelowy aktywny, email o zmianie.

S13. Webhooki: idempotentność, chargeback, rozbieżność (AC11)

- **Duplikat**: w Panelu tpay → narzędzie testu powiadomień → wyślij tę samą notyfikację 2×. Weryfikacja: DB `webhook_events.processing_status` drugi raz = `duplicate`; subskrypcja/faktura bez zmian (brak podwójnych efektów).
- **Chargeback**: jeśli panel sandbox udostępnia chargeback na transakcji — wykonaj na opłaconej z S1. Weryfikacja: `payment_transactions.status =`

`'charged_back'` + flaga ręcznej weryfikacji; **bez automatycznego downgrade'u** subskrypcji.
- **Odporność**: `GET /api/v1/billing/webhooks/health` (admin) — liczniki processed/duplicate rosną, brak serii błędów.

S14. Limity, progi 80/90/100, rate limiting (AC13) — opcjonalnie

- Limity: przy Freemium po grace (S11) — 11. analiza → 429 + CTA.
- Progi: podnieś licznik SQL-em (§4.4: `value=8` przy limicie 10) → wywołaj zadanie progowe:

```
docker compose exec backend celery -A app.workers.celery_app call \
  app.workers.tasks.check_thresholds.check_usage_thresholds
```

  → dzwonek in-app + email (logi).
- Rate limit: 101 żądanie/min z jednego konta → 429 z nagłówkami `X-RateLimit-*` (np. pętla `curl`).

4. Weryfikacje SQL / przyspieszanie czasu

Wykonanie: `docker compose exec postgres psql -U <user> -d <db> -c "..."`.

```sql
-- 4.1 Stan billingowy tenanta
SELECT s.status, s.cancel_at_period_end, s.current_period_end,
         s.next_charge_at,
       s.dunning_attempt, s.dunning_next_retry_at, s.grace_until, sp.name
         AS plan
FROM subscriptions s JOIN subscription_plans sp ON sp.id = s.plan_id
WHERE s.tenant_id = (SELECT id FROM tenants WHERE slug = '<slug>');

-- 4.2 Transakcje + webhooki + faktury
SELECT hidden_description, status, amount, paid_amount, test_mode,
       metadata->>'source' AS source, created_at FROM
         payment_transactions ORDER BY created_at DESC LIMIT 10;
SELECT event_uid, type, processing_status, error_message FROM
         webhook_events ORDER BY created_at DESC LIMIT 10;
SELECT invoice_number, status, issued_at FROM invoices ORDER BY issued_at
         DESC LIMIT 5;

-- 4.3 Przyspieszanie czasu (uwaga: tylko środowisko testowe!)
UPDATE subscriptions SET next_charge_at = now() - interval '1 hour'
         -- wymagalność odnowienia
WHERE tenant_id = (SELECT id FROM tenants WHERE slug = '<slug>');
UPDATE subscriptions SET dunning_next_retry_at = now() - interval '1
         hour'  -- (b) skróć cooldown dunningu
WHERE dunning_next_retry_at IS NOT NULL;
UPDATE subscriptions SET current_period_end = now() - interval '1 hour',
         -- (c) wykonaj downgrade / grace
    grace_until = now() - interval '1 hour'
WHERE tenant_id = (SELECT id FROM tenants WHERE slug = '<slug>');

-- 4.4 Licznik analiz na 80% limitu Freemium (próg powiadomień)
INSERT INTO usage_counters (id, tenant_id, counter_type, value,
         limit_value, period_start, period_end)
VALUES (gen_random_uuid(), (SELECT id FROM tenants WHERE slug='<slug>'),
         'analyses', 8, 10,
        date_trunc('month', now()), date_trunc('month', now()) + interval
         '1 month')
ON CONFLICT DO NOTHING;
```

5. Czyszczenie stanu między scenariuszami

- Ceny: `python scripts/set_sandbox_prices.py --reset` + restart.
- Tenant testowy: ustaw na Freemium (`UPDATE subscriptions s SET plan_id = (SELECT id FROM subscription_plans WHERE name='freemium'), status='active', cancel_at_period_end=false, next_charge_at=NULL, dunning_attempt=0, dunning_next_retry_at=NULL, grace_until=NULL, previous_plan_id=NULL WHERE s.tenant_id=...; UPDATE tenants SET plan='freemium' WHERE ...`) lub użyj `backend/scripts/reset_demo.py`.
- Ewentualne nowy tenant = czysty Freemium (najprostszy reset).

6. Kryteria zaliczenia (całość)

1. Każdy scenariusz S1–S13 (S14 opcjonalny) zakończony oczekiwanym wynikiem — w szczególności **strony błędów** (S4/S5/S6/S7) pokazują komunikat + ponowienie, tenant **zostaje na planie wyjściowym**.

2. `webhook_events` bez statusów `error` (poza celowo wywołanymi); health webhooków bez alertu.
3. Każda opłacona transakcja ma fakturę; `test_mode` = true wszędzie.
4. Po sesji: ceny zresetowane, tenant testowy na Freemium.